

Predicting GPU Frequencies to Reduce Energy Waste

ADOMAS PUSLYS, University of Twente, The Netherlands

GPU workloads are often executed at frequencies that provide limited additional performance while increasing energy consumption. Dynamic Voltage and Frequency Scaling (DVFS) can be used to programmatically lower clock frequencies and reduce this waste. However, kernels differ in how they respond to frequency changes and therefore prefer different clock settings. This work evaluates prediction of GPU clock settings to reduce energy waste in kernel sequences.

To this end, we conduct empirical, proof-of-concept research using a set of 65 kernels from PolyBench-GPU and Rodinia on an NVIDIA RTX 3080 Ti GPU. We exhaustively measure each kernel *in isolation*, across 30 clock settings, using a 6×5 grid of core and memory clock settings. We use the measurements to construct *local* and *global* oracles. We then train (1) Random Forest, (2) Extremely Randomized Trees, and (3) XGBoost regressors to predict clock settings using clock pairs, static PTX and launch configuration features, and runtime profiling features collected with Nsight Compute.

Our empirical analysis shows maximum energy savings by the global oracle of 13.40% to 20.09% across slowdown bounds from 0% to 5%. Our best predictions recover roughly 77–88% of the global-oracle savings.

We further propose kernel grouping to make clock selection applicable when individual kernels are shorter than the frequency-switching delay. At the 0% slowdown bound, grouping reduces clock changes from 64 to 22 while preserving 13.38% energy saving, compared with 13.40% for the per-kernel global oracle.

Additional Key Words and Phrases: GPU DVFS, energy waste, frequency prediction, kernel sequences, runtime and energy modeling, kernel grouping

1 INTRODUCTION

As high-performance computing (HPC) and artificial intelligence (AI) workloads require increasingly larger computational resources, the power demand for GPU computing has grown substantially [7, 22]. This high power draw increases both operational expenses and cooling demands in datacenters [7]. Crucially, a significant portion of this energy does not translate to performance directly. Because default hardware management policies often prioritize peak throughput, workloads frequently execute at higher core and memory clock frequencies than necessary [22]. This results in what Chung et al. call *energy bloat*: energy consumption that can be reduced without slowing down the workload [7]. In this work, we focus on systematic approaches to reduce this energy bloat.

Dynamic Voltage and Frequency Scaling (DVFS) can reduce energy waste by adjusting GPU clock frequencies [13]. While lowering frequencies decreases instantaneous power draw, it can also extend execution time, creating a direct trade-off between power and performance. On modern GPUs, the primary DVFS controls are the core clock, which determines streaming multiprocessor execution speed, and the memory clock, which scales memory bandwidth. Selecting optimal frequencies for these clocks is difficult because the best

(core,memory) frequency pair depends on the kernel’s instructions, launch configuration, runtime memory behavior, and the specific GPU architecture [1, 20]. In this work, we build models that predict the most (core,memory) frequency pair(s).

Many solutions that improve the energy efficiency of GPU applications use DVFS to minimize the Energy-Delay Product (EDP) or its variants [12]. However, these metrics may allow unconstrained slowdowns to save energy [16]. Instead, the *compute waste reduction* objective of Spaan et al. [16] minimizes energy consumption while allowing only limited runtime slowdown. As this is more realistic for performance-sensitive workloads, our models are designed to select the (core,memory) frequency pair(s) that minimize(s) waste.

For complex, multi-kernel applications, selecting and applying a single DVFS configuration for the entire application (i.e., *coarse-grain DVFS*) may miss energy-saving opportunities due to the diversity of executed kernels and their inputs. Prior work [16] showed that combining *fine-grained DVFS* with *global-performance degradation limits*, i.e., exploiting kernel diversity by adjusting the settings for *individual kernels* towards *end-to-end performance preservation*, allows for more optimization opportunities. Thus, in our work, we tackle the additional challenge of modeling and selecting *local DVFS configurations* for *aggregated, global* energy savings and performance degradation bounds.

One approach to identifying optimal frequency configurations is *autotuning*: (exhaustively) measuring each kernel’s energy consumption and performance across a grid of core and memory clock frequencies [14, 17]. While useful for establishing an *oracle*, i.e., the best measured configuration within the evaluated frequency grid, this process is too expensive to repeat for each new workload and new system [8]. Instead, model-based prediction offers a lower-cost alternative: runtime and energy scaling can be estimated from static code features, launch-configuration features, and runtime-profiling features, allowing frequencies to be selected without measuring the full DVFS space [3, 8, 11].

In this work, we address the challenge of *predicting per-kernel (core, memory) frequencies under performance-loss constraints for multi-kernel GPU applications*. Therefore, we investigate whether GPU core and memory frequency settings can be predicted for kernel sequences within a pre-defined slowdown bound.

This leads to the following research question:

RQ: To what extent can GPU core and memory clock settings be predicted for kernel sequences to reduce energy waste while preserving performance?

We answer this question through three sub-questions:

SQ1: How much energy waste can we reduce using a local and global oracle clock selection?

SQ2: Which feature sets and models best predict execution time and energy under different clock settings?

SQ3: How much energy waste can the best predicted clocks reduce compared to the (global) oracle?

Our approach has the following steps. First, we measure each kernel’s performance and energy consumption in isolation, across a grid of feasible GPU core and memory clock settings. Next, we use these measurements to construct local and global oracle baselines. Both select the lowest-energy (core,memory) clock pair per kernel from the measured grid. The local oracle selects the clock pair for each kernel separately. The global oracle selects a set of clock pairs for the full sequence, where individual kernels may slow down as long as the total sequence runtime stays within the slowdown bound. We then train predictive models that use kernel and clock information to select clock pairs. Finally, we evaluate the selected pairs using measured runtime and energy, and compare their savings against the local and global oracle baselines.

We evaluate this approach on a sequence of 65 kernels from PolyBench-GPU and Rodinia on an NVIDIA RTX 3080 Ti GPU. Across slowdown bounds from 0% to 5%, the global oracle saves 13.40% to 20.09% energy relative to automatic clocks. The best predictions save 10.42% to 17.61% energy, recovering roughly 77-88% of the global oracle savings.

The main contributions of our research are as follows:

- We construct a GPU DVFS dataset by measuring isolated kernels across core and memory clock frequencies, and use it to define local and global oracles.
- We evaluate prediction-based GPU clock selection using PTX, launch-configuration, runtime-profiling, and combined feature sets, recovering 77–88% of the global-oracle energy savings.
- We introduce kernel grouping to account for frequency-switching delay. At the 0% slowdown bound, grouping reduces clock changes from 64 to 22 while preserving 13.38% energy savings.

2 BACKGROUND

This section introduces the concepts used throughout the paper: GPU DVFS and its clock domains (§2.1), static and runtime kernel characteristics (§2.2), and regression models for runtime and energy prediction (§2.3).

2.1 GPU DVFS and Clock Domains

Dynamic Voltage and Frequency Scaling (DVFS) changes processor clock frequency, often together with the corresponding supply voltage. Lowering the frequency usually reduces instantaneous power, but it can also increase execution time. Therefore, total energy does not necessarily decrease: energy is reduced only when the power saving is large enough to compensate for the longer runtime [13].

On modern NVIDIA GPUs, users typically do not directly control voltage. Instead, clock frequencies can be requested through vendor tools and management APIs such as `nvidia-smi`¹ and NVML².

GPU clocks can either be managed automatically or requested manually. Under automatic clock management, the GPU selects core and memory frequencies according to workload demand, power limits, temperature limits, and idle state³. Manual DVFS instead

requests specific core or memory clock frequencies, allowing experiments to evaluate how execution time and energy change under controlled settings [16].

2.2 Static and Runtime GPU Kernel Characteristics

GPU kernels are functions launched by the CPU and executed on the GPU.

Static kernel characteristics are features that can be extracted before execution. GPU kernel behavior depends on both the code being executed and the way the kernel is launched. CUDA source code provides program structure and launch information, such as grid dimensions, block dimensions, and shared-memory arguments. Before final machine code generation, CUDA kernels can be represented in Parallel Thread Execution (PTX), an intermediate representation from which features such as opcode categories, memory-space usage, data types, branches, and register declarations can be extracted [3, 11]. Because these features can be extracted from source code and PTX files, they can be collected without profiling the kernel.

Runtime kernel characteristics describe behavior observed during execution, such as cache activity, memory bandwidth utilization, achieved occupancy, stalls, and pipeline utilization. These features can capture behavior that is not obtainable from static code, but require additional profiling before prediction.

2.3 Lightweight Regression Models

Regression models predict continuous numerical outputs from input features. In GPU performance energy modeling, the inputs can include clock settings and kernel characteristics, while the outputs can represent quantities such as runtime, power, or energy.

Tree-based regression models model nonlinear relationships between input features and output values without requiring extensive feature scaling [15]. A *decision tree* does this by repeatedly splitting the data using feature thresholds and assigning a prediction to each final leaf. Since a single decision tree can overfit noisy measurements, ensemble methods combine many trees to increase robustness.

Specifically, *random forests* [4] train many decision trees on different samples of the training data and average their predictions. *Extremely Randomized Trees* [9] also average many decision trees, but choose split thresholds randomly instead of searching for the best threshold at each split. *Gradient-boosted trees*, such as XGBoost [6], add trees sequentially, where each new tree is trained to reduce the remaining prediction error.

3 RELATED WORK

This work builds on prior research on GPU DVFS, performance and energy prediction, and fine-grained clock selection. We review existing prediction methods based on static code and launch features (§3.1), runtime and hybrid DVFS prediction methods (§3.2), DVFS selection (§3.3), and GPU frequency-switching costs (§3.4).

3.1 Prediction from Static Features

Several GPU modeling approaches predict kernel behavior from information available before execution, such as code features, launch

¹<https://docs.nvidia.com/deploy/nvidia-smi/>

²<https://docs.nvidia.com/deploy/nvml-api/nvml-api-reference.html>

³https://docs.nvidia.com/deploy/nvml-api/group__nvmlClocksEventReasons.html

parameters and requested clock settings. Unlike runtime methods, these approaches do not require kernel profiling before prediction.

Braun et al. predict GPU kernel execution time and power using PTX-derived features and launch parameters, evaluated across several benchmark suites and NVIDIA GPUs [3]. Their work demonstrates that static code and launch features can predict runtime and power, but it does not address core-memory frequency selection.

Fan et al. predict speedup and normalized energy for different GPU core and memory frequency settings using static instruction features and frequency values [8]. The predicted values are used to derive Pareto-optimal frequency configurations for each kernel. Their approach derives per-kernel configurations, rather than optimizing clock choices globally across a kernel sequence under a shared runtime limit.

Guerreiro et al. predict execution time, power, and energy scaling factors for different GPU frequency settings using a learned PTX representation combined with the target core and memory frequency pair [11]. This captures instruction order, but uses a more complex model and evaluates frequency selection mainly for individual applications.

3.2 Runtime and Hybrid DVFS Prediction

Runtime-based methods use hardware counters or utilization metrics collected during execution. Wang and Chu estimate GPU kernel execution time under different core and memory frequencies using microbenchmarks and performance counters collected at a baseline frequency [19]. Ali et al. similarly use GPU utilization metrics collected at a single frequency to estimate power and execution time across the DVFS space, and then select an energy-performance frequency trade-off using EDP-based objectives [2].

DSO combines runtime metrics from NVIDIA DCGM with static PTX-derived features [20]. Instead of directly predicting runtime or energy ratios, it uses a machine learning model to predict the parameters of a theoretical GPU DVFS model. These parameters are then used to choose a core-memory frequency setting. DSO reports lower prediction error than using PTX or DCGM features alone and achieves about 19% energy savings on a Tesla V100 while keeping performance loss within 5%.

3.3 Fine-Grained and Sequence-Level DVFS

Spaan et al. study fine-grained DVFS for LLM training, where the workload is a sequence of GPU kernels with different sensitivity to clock frequencies [16]. They define a compute-waste objective: minimizing energy while bounding total runtime loss. Their *local* policy selects a clock setting for each operation independently, while the *global* policy selects clock settings for the full sequence, allowing extra runtime in one kernel to be compensated elsewhere.

3.4 Frequency-Switching Costs

Fine-grained GPU DVFS depends not only on which clock settings are energy-efficient, but also on how expensive it is to change between them. A policy that changes clocks too frequently may lose part of its energy benefit to frequency-switching overhead. Wang and Chu report GPU frequency-switching latency of 10–15 ms on a Tesla V100 using NVML application clocks [19]. Velicka et al. show

that GPU switching latency varies strongly by device and frequency pair, with measured transitions across RTX Quadro 6000, A100-SXM4, and GH200 ranging from a few milliseconds to hundreds of milliseconds [18].

Overall, prior work shows that GPU runtime, power, and energy can be predicted from static, runtime, and hybrid features. It also shows that per-kernel clock choices can be evaluated globally, so that extra runtime in one kernel can be compensated by faster choices elsewhere in the sequence. Our research uses predicted runtime and energy ratios for local and global clock selection on benchmark kernel sequences, and evaluates grouping as a way to reduce frequency-switching overhead.

4 PREDICTION FRAMEWORK

This section introduces our methodology to select and evaluate clock settings. We first define the local and global oracles (§4.1). Next, we present the prediction targets and frequency-selection procedure (§4.2) and the policy evaluation methodology (§4.3).

4.1 Oracle Construction

For each kernel k and clock pair c , the measurement dataset contains the runtime $T(k, c)$ and energy $E(k, c)$ per launch. The automatic-clock measurement of the same kernel is used as the baseline, denoted $T(k, \text{auto})$ and $E(k, \text{auto})$.

Two oracle policies are considered. The local oracle selects one clock pair for each kernel independently. For slowdown bound ϵ , the selected clock pair must satisfy

$$T(k, c) \leq (1 + \epsilon)T(k, \text{auto}).$$

Among all valid clock pairs, including the automatic-clock baseline, we select the one with the lowest measured energy.

The global oracle instead selects one clock pair per kernel while applying the slowdown constraint to the full kernel sequence. For a sequence of kernels K , it solves

$$\min_{c_k} \sum_{k \in K} E(k, c_k)$$

subject to

$$\sum_{k \in K} T(k, c_k) \leq (1 + \epsilon) \sum_{k \in K} T(k, \text{auto}).$$

The objective minimizes total measured energy over the sequence, while the constraint limits total sequence runtime. This allows slowdown in one kernel to be compensated by faster/unchanged execution in other kernels, while the total runtime remains within the specified bound. The problem is solved as a mixed-integer linear program, with one selected clock pair for each kernel.

4.2 Prediction and Frequency Selection

The prediction models estimate runtime and energy ratios relative to automatic clocks. For a kernel k and clock pair c , the targets are

$$r_t(k, c) = \frac{T(k, c)}{T(k, \text{auto})}, \quad r_e(k, c) = \frac{E(k, c)}{E(k, \text{auto})}.$$

Separate models are trained for r_t and r_e . Each training row represents one kernel under one core-memory clock pair. The input

features include the clock pair together with the feature sets described in Section 5. Automatic-clock rows are not used for training, but are included during selection with $r_t = r_e = 1$.

We use Random Forests, Extremely Randomized Trees, and XGBoost regressors because the data is tabular and the relation between clock settings, kernel features, runtime, and energy is expected to be nonlinear. Separate regressors are trained for runtime and energy so that frequency selection can use both predicted quantities in the optimization step.

We train the regressors on log-transformed targets, $\log(r_t)$ and $\log(r_e)$, so prediction errors are scaled relative to the magnitude of the ratios. Before clock selection, we exponentiate the predicted log-values to obtain predicted ratios, $\hat{r}_t$ and $\hat{r}_e$. We multiply these ratios by the automatic-clock runtime and energy of the same kernel:

$$\hat{T}(k, c) = \hat{r}_t(k, c)T(k, \text{auto}), \quad \hat{E}(k, c) = \hat{r}_e(k, c)E(k, \text{auto}).$$

The predicted local policy applies the same per-kernel rule as the local oracle, but uses $\hat{T}$ and $\hat{E}$ instead of measured values. The predicted global policy solves the same problem as the global oracle, but minimizes predicted total energy under a predicted total runtime constraint.

4.3 Evaluation Methodology

To evaluate the results of the prediction, we compare against the measured data. Specifically, we use the predicted values only to choose clock pairs. After the choices are made, each selected kernel-clock pair is compared against its measured runtime and energy in the dataset. The total runtime and total energy are then summed over all kernels and compared with the automatic-clock baseline.

5 EXPERIMENTAL SETUP

This section describes how the measurement and prediction datasets are constructed. We first describe the measurement protocol (§5.1). Then, we explain the benchmark selection (§5.2) and kernel isolation procedure (§5.3). Next, we define the tested frequencies and effective-clock aggregation (§5.4). Finally, we describe feature extraction (§5.5) and model training (§5.6).

5.1 Hardware and Measurement Protocol

All measurements are performed on an NVIDIA RTX 3080 Ti GPU. The benchmarks are compiled with CUDA 13.2 (nvcc V13.2.51). GPU frequencies are controlled with `nvidia-smi`, while energy is measured through NVML using `nvmlDeviceGetTotalEnergyConsumption`. Runtime is measured using CUDA events.

Before measuring the kernels of a workload, the GPU is warmed up at maximum core and memory clocks for 180 seconds, aiming to reach a stable temperature of approximately 77°C.

For each isolated kernel, measurements are collected under automatic and manually requested fixed clock pairs. Fixed-clock measurements request a specific core-memory clock pair using `nvidia-smi`. Automatic-clock measurements reset the GPU clocks to its default automatic policy, allowing the GPU to select clocks dynamically during execution.

Before measurement, the isolated kernel is warmed up for 25 seconds at the selected clock pair. NVIDIA onboard energy measurements can be affected by the sensor update period and internal

averaging window, especially for short GPU executions [21]. Therefore, the selected kernel is launched repeatedly for 5 seconds to form a longer measurement window. CUDA events are recorded before and after this window to measure runtime, and `nvmlDeviceGetTotalEnergyConsumption` is read before and after the same window to measure energy.

5.2 Benchmark Selection

The benchmark set is selected from PolyBench-GPU and Rodinia, which are commonly used GPU benchmark suites covering structured kernels, heterogeneous applications, and throughput-computing workloads [5, 10]. Together, they provide a mix of dense linear algebra, stencil, graph, dynamic-programming, image-processing, and memory-intensive workloads.

We include the benchmarks that compile with the available CUDA environment and contain kernels that can be isolated into repeatable measurement executables. We exclude benchmarks that depend on legacy CUDA APIs, unavailable external libraries, or whose state cannot be cleanly reset for repeated isolated measurements.

After benchmark filtering and kernel isolation, the dataset contains 65 kernels: 38 from PolyBench-GPU and 27 from Rodinia.

5.3 Kernel Isolation

We transform the original benchmarks into isolated measurement executables, with each executable measuring a single CUDA kernel. The original input initialization, problem size, kernel code, and grid/block configuration are kept as close as possible to the original benchmark. Setup and validation code is kept outside the measured region, so the recorded runtime and energy correspond only to repeated launches of the isolated kernel. For kernels that depend on outputs from earlier kernels, we reconstruct the required input arrays before measurement.

5.4 Frequency Grid and Effective Clocks

We measure each isolated kernel across a grid of requested core and memory clock settings. The manually requested core clocks are 210, 630, 1050, 1470, 1890, and 2100 MHz. The manually requested memory clocks are 405, 810, 5001, 9251, and 9501 MHz. In addition, we measure each kernel under automatic clocks, which we further use as the baseline for normalization, oracle construction, and evaluation.

For fixed clock measurements, we store the core and memory clocks observed through NVML. We refer to these values as *effective clocks*. This distinction is necessary because the highest requested core clock is not available as an effective clock on this GPU. Although 2100 MHz can be requested, NVML reports lower effective core clocks for those runs, typically 1965 MHz. When several requested clock pairs lead to the same effective clock pair, their measured runtime and energy values are averaged into one record.

After the aggregation, the final dataset contains 1,666 measured kernel-clock records: 974 from 38 PolyBench-GPU kernels and 692 from 27 Rodinia kernels. The dataset also contains 65 automatic clock baseline records, one per kernel, used as the reference for runtime and energy ratios.

5.5 Feature Extraction

The prediction dataset uses four groups of input features: effective clock values, PTX features, launch features, and runtime features.

We extract PTX features from the PTX code generated for each isolated kernel. The parser counts PTX instructions, instruction types, data types, memory space usage, static memory access estimates, and virtual register declarations. These counts are also combined into grouped features, including arithmetic, integer, logic, memory, global memory, branch, predicate, conversion, synchronization, special operation, ratio, and arithmetic intensity features. We use normalized versions of these features to reduce dependence on PTX file size.

We extract launch features from the CUDA kernel launch configuration. These include grid dimensions, block dimensions, the number of CTAs, threads per CTA, warps per CTA, and dynamic shared memory size when available. Theoretical occupancy features are estimated from the launch features, approximate PTX register counts, dynamic shared memory, and RTX 3080 Ti hardware limits. These occupancy values are static estimates (i.e., not the measured runtime occupancy).

We extract runtime features from Nsight Compute measurements. These features describe observed kernel behavior, including memory activity, cache behavior, achieved occupancy, and pipeline utilization. Unlike PTX and launch features, runtime features require profiling before prediction. Across the 65 measured kernels, one Nsight Compute profile takes a median of approximately 3 seconds.

We construct the prediction dataset by joining the PTX, launch, and runtime feature vectors for each kernel with each measured effective clock pair in a single record. Thus, each training record represents one kernel under one effective core-memory clock pair.

The evaluated feature sets are: PTX, launch, runtime, PTX with launch, PTX with runtime, launch with runtime, and PTX with launch with runtime. All feature sets also include the effective core and memory clock values.

5.6 Model Training and Validation

We trained Random Forest, Extremely Randomized Trees, and XGBoost regressors using leave-one-kernel-out validation. For each fold, we held all measurements of one kernel across all effective clock pairs out for testing, while the model is trained on all remaining kernels. This approach prevents the model from seeing the same kernel’s PTX and launch features during both training and testing.

We train separate models for runtime and energy ratios. The models are trained on the logarithm of the ratio targets, and predictions are converted back to ratio scale before clock selection. Random Forest and Extremely Randomized Trees use 512 trees, a minimum leaf size of 2, and a fixed random seed. XGBoost is trained with 512 boosting rounds using the same leave-one-kernel-out folds.

We also measure model overhead during leave-one-kernel-out validation. Each fold trains on 64 kernels and predicts the held-out kernel. Across all folds and feature sets, median training time is 1.04 seconds per fold. Since training is performed offline, the more relevant deployment cost is inference. Median inference time per prediction is 0.29 ms for XGBoost and $\approx 10ms$ for Random Forest and Extremely Randomized Trees. Clock selection evaluates one

prediction for each candidate clock pair, so per-kernel inference time scales with the number of clock pairs.

We report prediction accuracy using mean absolute error (MAE), mean absolute percentage error (MAPE), and the 90th percentile absolute percentage error. We evaluate the selected clock pairs against the measured runtime and energy values, as described in Section 4.3.

6 RESULTS

This section reports the main experimental results. We present the local and global oracles (§6.1), the prediction accuracy evaluation (§6.2), and the predicted clock selection impact (§6.3).

6.1 Oracle Results

Table 1 compares the local and global oracles for all kernels. Both oracles select one clock pair per kernel from the measured effective-clock grid, but they apply the slowdown bound differently: The local oracle applies the bound per kernel, while the global oracle applies it to the total runtime of the full kernel sequence.

Table 1. Oracle savings relative to automatic GPU clocks.

Oracle	Bound	Energy savings	Runtime change
Local	0%	12.13%	-0.67%
	1%	12.16%	-0.67%
	2%	15.50%	0.45%
	5%	19.49%	2.09%
Global	0%	13.40%	0.00%
	1%	16.38%	1.00%
	2%	19.18%	2.00%
	5%	20.09%	2.94%

We observe that the local oracle is stricter. This is because each kernel must satisfy its own slowdown bound. A lower-energy clock pair is only valid if it remains within that kernel’s slowdown bound. As a result, the local oracle does not necessarily use the full allowed runtime budget. Instead, the global oracle obtains higher savings than the local oracle at all bounds. The difference is largest at the 2% bound, where the global oracle gains 3.68 percentage points over the local oracle.

6.2 Prediction Accuracy

Tables 2 and 3 report leave-one-kernel-out prediction accuracy for energy-ratio and runtime-ratio prediction. All feature sets include the effective core and memory clock pair. Runtime refers to selected Nsight Compute runtime counters, while PTX and launch refer to static PTX-derived features and launch-configuration features.

Table 2. LOKO prediction accuracy for energy-ratio prediction.

Features	Model	n_f	MAE	MAPE (%)	P90 APE (%)
PTX	RF	59	0.280	21.50	51.21
PTX	ET	59	0.270	20.97	50.29
PTX	XGB	59	0.268	21.89	52.71
Launch	RF	19	0.203	15.76	38.51
Launch	ET	19	0.212	16.17	40.01
Launch	XGB	19	0.198	15.56	34.43
Run	RF	162	0.119	8.66	20.71
Run	ET	162	0.107	8.07	21.07
Run	XGB	162	0.103	7.80	18.24
PTX+Launch+Run	RF	226	0.127	9.16	22.53
PTX+Launch+Run	ET	226	0.109	8.12	21.63
PTX+Launch+Run	XGB	226	0.113	8.43	20.87

Table 3. LOKO prediction accuracy for Run-ratio prediction.

Features	Model	n_f	MAE	MAPE (%)	P90 APE (%)
PTX	RF	59	1.048	27.88	62.47
PTX	ET	59	1.081	27.78	62.63
PTX	XGB	59	1.014	29.33	57.87
Launch	RF	19	0.650	18.19	43.49
Launch	ET	19	0.682	18.40	45.82
Launch	XGB	19	0.621	18.84	44.89
Run	RF	162	0.359	9.36	24.63
Run	ET	162	0.325	8.97	25.30
Run	XGB	162	0.322	8.71	22.36
PTX+Launch+Run	RF	226	0.389	9.85	24.23
PTX+Launch+Run	ET	226	0.327	8.28	22.37
PTX+Launch+Run	XGB	226	0.360	9.04	23.27

The prediction results show that runtime features are the most reliable input source for both targets: the best runtime-based models reach about 8–9% MAPE, compared with 16–19% for launch-only features and 21–29% for PTX-only features. Adding PTX or launch features on top of runtime features gives little additional benefit, as Run and PTX + launch + run both remain in roughly the same error range. This suggests that measured runtime behavior is more informative for clock sensitivity than features available prior to execution.

6.3 Predicted Global Clock Selection

The models predict runtime and energy ratios for each candidate clock pair, and the optimizer uses these predictions to select one clock pair per kernel. We compute the reported savings from the measured runtime and energy of the selected clock pairs, not from the predicted values.

Figure 1 shows all predicted global selections across the evaluated slowdown bounds. The global oracle is shown as an upper reference, while the best within-bound prediction is highlighted at each bound. Points marked as bound violations correspond to selections whose measured runtime exceeds the allowed slowdown.

Table 4 summarizes the best within-bound prediction at each slowdown limit. As expected, stricter bounds still limit the energy savings opportunities: we can save 10.42% energy for the 0% bound and 17.61% for the 5% bound. Across the evaluated bounds, the best within-bound predictions recover about 77–88% of the oracle saving.

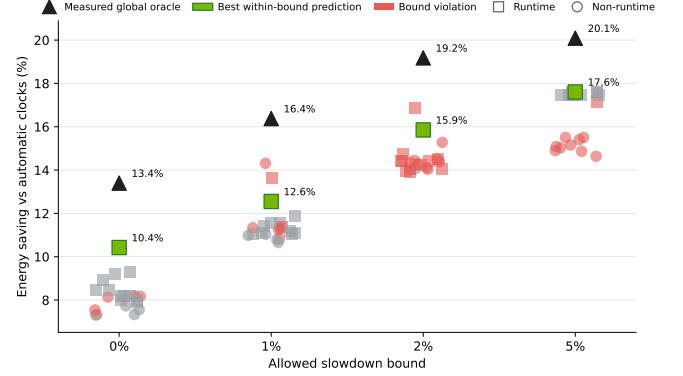


Fig. 1. Measured energy savings of predicted global clock selections across slowdown bounds. The global oracle and the best within-bound prediction are highlighted for each bound.

The gap decreases as the slowdown bound becomes more relaxed, because the optimizer has more room to select lower-energy clock pairs without exceeding the measured runtime constraint.

Table 4. Best within-bound predicted selections compared with the measured global oracle.

Bound	Features	Model	Oracle saving	Measured saving	Runtime change
0%	PTX+Run	XGB	13.40%	10.42%	−0.08%
1%	PTX+Launch+Run	XGB	16.38%	12.55%	0.48%
2%	PTX+Launch+Run	XGB	19.18%	15.85%	1.67%
5%	PTX+Run	ET	20.09%	17.61%	2.90%

Table 5 reports the full set of predicted selections at the 0% slowdown bound. Shaded rows indicate selections whose measured runtime is above the allowed limit.

At the 0% bound, the best within-bound prediction uses PTX + launch + runtime, saving 9.19% energy, and decreasing runtime by 0.62%. Several other runtime-containing policies also remain within the bound. In contrast, some PTX-only selections exceed the measured runtime limit.

At relaxed slowdown bounds, runtime-containing feature sets generally remain closer to the allowed runtime range, while runtime-free feature sets more often exceed the bound.

Table 5. Measured outcomes of predicted clock selections at 0% slowdown. Shaded rows exceed the measured runtime bound.

Features	Model	Measured saving	Runtime change
PTX	RF	7.73%	−0.12%
PTX	ET	8.14%	0.43%
PTX	XGB	7.33%	0.29%
Launch	RF	7.31%	−0.34%
Launch	ET	7.33%	−0.30%
Launch	XGB	7.56%	−0.18%
Runtime	RF	8.18%	−0.07%
Runtime	ET	9.32%	−0.59%
Runtime	XGB	8.17%	−0.09%
PTX+launch+runtime	RF	8.00%	−0.10%
PTX+launch+runtime	ET	8.45%	−0.08%
PTX+launch+runtime	XGB	9.19%	−0.62%

Figure 2 compares the selected clock pairs for the global oracle and the best within-bound predicted selections. In both cases, core-clocks cluster around the 1470–1890 MHz region, while memory-clock choices remain spread across the measured grid.

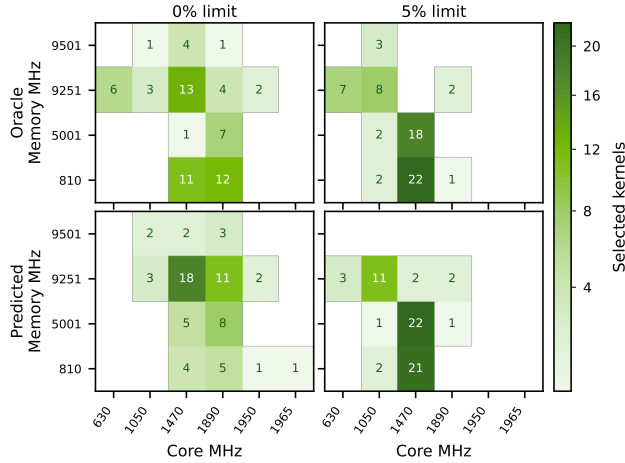


Fig. 2. Effective clock pairs selected by the global oracle and the best within-bound predicted global policy at the reported slowdown limits. Each cell shows the number of selected kernels.

7 KERNEL GROUPING

This section motivates kernel grouping (§7.1), presents our grouping method (§7.2), and reports its results (§7.3).

7.1 Grouping Motivation

The oracle and prediction results in Section 6 assume that clock pairs can be selected independently for each kernel. In practice, a requested clock pair is not applied instantly. If kernels are shorter than the time needed to change clocks, a separate clock change cannot be completed for each kernel. Therefore, such kernels should be grouped under a shared clock pair.

Prior measurements show that GPU frequency-switching latency depends on the GPU, target frequency, and initial frequency [18]. Velicka et al. define this latency as the time between issuing a frequency-change request from the CPU and the GPU reaching the target frequency. For example, on A100-SXM4, they report worst-case switching times of 7.413 ms for 1350 → 1260 MHz, but 22.716 ms for 1125 → 795 MHz. Reported switching times range from a few milliseconds on A100-SXM4 to hundreds of milliseconds on RTX Quadro 6000 and GH200 [18].

Figure 3 shows that many measured kernels are much shorter than these switching times. Automatic-clock runtime ranges from 5.88 μs to 1.35 s, with a median of 0.151 ms. In total, 47 of the 65 kernels execute in less than 1 ms. Thus, for millisecond-scale switching times, most kernels are shorter than a single clock transition, so per-kernel clock changes are not possible.

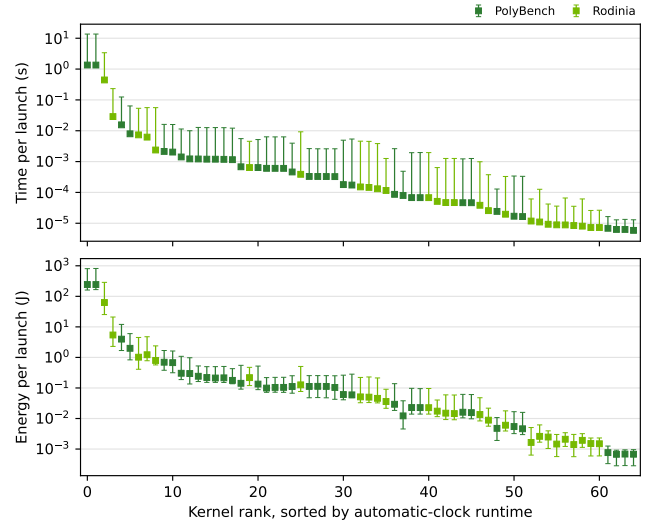


Fig. 3. Per-kernel runtime and energy distribution. Markers show values at automatic GPU clocks, while vertical bars show the minimum and maximum values observed across the measured fixed-clock configurations. Kernels are sorted by automatic-clock runtime. Marker color indicates the benchmark suite.

7.2 Grouped Clock Selection

To account for switching delay, we propose applying clock settings to groups of kernels instead of to individual kernels. Each group contains one or more kernels and runs under one shared clock pair. For a group G and clock pair c , the group runtime and energy are computed from the measured per-kernel values:

$$T(G, c) = \sum_{k \in G} T(k, c), \quad E(G, c) = \sum_{k \in G} E(k, c).$$

After groups are formed, one clock pair is selected for each group. For a set of groups $\mathcal{G}$, the grouped policy minimizes total measured energy:

$$\min_{c_G} \sum_{G \in \mathcal{G}} E(G, c_G)$$

subject to the same global slowdown bound used by the global oracle:

$$\sum_{G \in \mathcal{G}} T(G, c_G) \leq (1 + \epsilon) \sum_{G \in \mathcal{G}} T(G, \text{auto}).$$

In a single sequential workload, grouping can only combine neighboring kernels. In a multi-workload setting, however, independent workloads can be interleaved. Each workload preserves its own kernel order, but kernels from different workloads can be scheduled together when they become *ready*. A kernel is ready if all earlier kernels in the same workload have already completed.

We define our grouping policy as follows. For a switching delay S , the scheduler maintains a ready set R . First, the scheduler selects the ready kernel with the largest automatic-clock runtime and uses its preferred clock pair as the target. It then forms a group by adding ready kernels whose preferred clocks match the target clock pair. If the group runtime is still below S and no exact match is ready, the

scheduler adds ready kernels with the closest preferred clocks. This continues until the accumulated automatic-clock runtime reaches at least S , or until no kernels remain.

For the grouping experiment, each benchmark workload is treated as an independent workload instance. The resulting schedule combines 31 workload instances containing 65 kernels. Of the 23 formed groups, 9 contain kernels from multiple workload instances, with 1 to 8 workload instances per group.

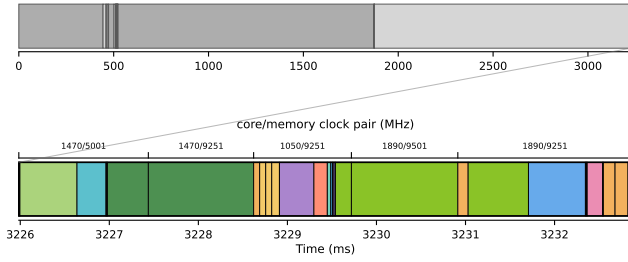


Fig. 4. Grouped schedule for the measured kernels. The top bar shows the full grouped schedule, and the bottom bar expands one region. Blocks represent kernels, with colors indicating workload. Tick marks above the expanded region indicate group boundaries where a new core-memory clock pair is selected. Clock-pair labels are shown as core/memory MHz.

Figure 4 shows the resulting grouped schedule. The expanded region illustrates that several kernels can execute under one selected clock pair before the next clock change.

7.3 Grouping Results

Table 6 reports the effect of grouping for an assumed switching delay of $S = 1$ ms at the 0% slowdown bound. The per-kernel global oracle selects one clock pair per kernel. The grouped global oracle selects one clock pair per group under the same global runtime constraint.

Table 6. Grouping result for $S = 1$ ms at the 0% slowdown bound.

Policy	Clock changes	Saving	Runtime change
Per-kernel global oracle	64	13.40%	0.00%
Grouped global oracle	22	13.38%	0.00%

With $S = 1$ ms, grouping reduces the number of clock changes from 64 to 22. The grouped global oracle saves 13.38% energy relative to automatic clocks, compared with 13.40% for the per-kernel global oracle. This corresponds to a loss of 0.02 percentage points while preserving the 0% global slowdown bound. This shows that, for the measured sequence, most of the global-oracle energy saving can be preserved even when clock changes are made only between groups.

8 IMPLICATIONS AND LIMITATIONS

We briefly discuss the main implications of the results (§8.1) and the main limitations of the proposed approach (§8.2).

8.1 Prediction Gap

The predicted global selections reduce energy at all measured slowdown limits, but a gap between the prediction and the global oracle remains: the predictions save 3-4% less than the oracle baselines.

The remaining gap is expected because the prediction models select clock pairs from estimated runtime and energy ratios, while the oracle selects from measured values. Even small prediction errors can change which clock pair appears best under the slowdown constraint. Separately, runtime-prediction errors also affect constraint reliability: runtime-free feature sets more often exceed the measured slowdown bound, while runtime-containing feature sets stay closer to the intended runtime range.

8.2 Scope and Limitations

Runtime features give the strongest predictions, but require Nsight Compute profiling before clock selection. In this dataset, one runtime profile takes a median of approximately 3 seconds per kernel.

For grouping, we use a fixed switching delay of $S = 1$ ms. In practice, switching time depends on the GPU and the specific clock transition [18]. Measuring switching latency on the RTX 3080 Ti would allow S to be chosen more precisely.

9 CONCLUSION

Our research investigated whether GPU core and memory clock settings can be predicted for kernel sequences under limited runtime slowdown. Kernels from PolyBench-GPU and Rodinia were isolated and measured on an NVIDIA RTX 3080 Ti across a grid of core and memory clock settings. These measurements were used to construct local and global oracles. Prediction models were then trained to estimate runtime and energy ratios from effective clocks, runtime counters, PTX-derived features, and launch-configuration features.

The measured global oracle saves 13.40%, 16.38%, 19.18%, and 20.09% energy at 0%, 1%, 2%, and 5% slowdown limits, respectively. The best within-bound predicted global selections save 10.42%, 12.55%, 15.85%, and 17.61% energy at the same limits, recovering roughly 77–88% of the measured global-oracle saving.

The results show that model-based global DVFS prediction can recover a large part of the oracle energy saving, especially when runtime information is available. Runtime features are substantially more accurate than PTX-only or launch-only features, while adding PTX or launch features on top of runtime features gives little additional benefit. Kernel grouping further shows that clock selection does not need to change after every kernel: for $S = 1$ ms, the grouped oracle uses 22 clock changes instead of 64 and saves 13.38% energy. Future work should evaluate the method on additional GPUs, denser frequency grids, larger application sequences, and measured switching delays on the target GPU.

REFERENCES

- [1] Gargi Alavani, Jineet Desai, Snehanishu Saha, and Santonu Sarkar. 2023. Program Analysis and Machine Learning-based Approach to Predict Power Consumption of CUDA Kernel. *ACM Trans. Model. Perform. Eval. Comput. Syst.* 8, 4, Article 10 (July 2023), 24 pages. <https://doi.org/10.1145/3603533>
- [2] Ghazanfar Ali, Mert Side, Sridutt Bhalachandra, Nicholas J. Wright, and Yong Chen. 2023. Performance-Aware Energy-Efficient GPU Frequency Selection using DNN-based Models. In *Proceedings of the 52nd International Conference on Parallel*

- Processing (Salt Lake City, UT, USA) (ICPP '23). Association for Computing Machinery, New York, NY, USA, 433–442. <https://doi.org/10.1145/3605573.3605600>
- [3] Lorenz Braun, Sotirios Nikas, Chen Song, Vincent Heuveline, and Holger Fröning. 2021. A Simple Model for Portable and Fast Prediction of Execution Time and Power Consumption of GPU Kernels. *ACM Trans. Archit. Code Optim.* 18, 1, Article 7 (Dec. 2021), 25 pages. <https://doi.org/10.1145/3431731>
 - [4] Leo Breiman. 2001. Random Forests. *Mach. Learn.* 45, 1 (Oct. 2001), 5–32. <https://doi.org/10.1023/A:1010933404324>
 - [5] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. 2009. Rodinia: A Benchmark Suite for Heterogeneous Computing. In *2009 IEEE International Symposium on Workload Characterization (IISWC)*. 44–54. <https://doi.org/10.1109/IISWC.2009.5306797>
 - [6] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (San Francisco, California, USA) (KDD '16)*. Association for Computing Machinery, New York, NY, USA, 785–794. <https://doi.org/10.1145/2939672.2939785>
 - [7] Jae-Won Chung, Yile Gu, Insu Jang, Luoxi Meng, Nikhil Bansal, and Mosharaf Chowdhury. 2024. Reducing Energy Bloat in Large Model Training. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (Austin, TX, USA) (SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 144–159. <https://doi.org/10.1145/3694715.3695970>
 - [8] Kaijie Fan, Biagio Cosenza, and Ben Juurlink. 2020. Accurate Energy and Performance Prediction for Frequency-Scaled GPU Kernels. *Computation* 8, 2, Article 37 (2020). <https://doi.org/10.3390/computation8020037>
 - [9] Pierre Geurts, Damien Ernst, and Louis Wehenkel. 2006. Extremely randomized trees. *Mach. Learn.* 63, 1 (April 2006), 3–42. <https://doi.org/10.1007/s10994-006-6226-1>
 - [10] Scott Grauer-Gray, Lifan Xu, Robert Searles, Sudhee Ayalasomayajula, and John Cavazos. 2012. Auto-tuning a High-Level Language Targeted to GPU Codes. In *2012 Innovative Parallel Computing (InPar)*. 1–10. <https://doi.org/10.1109/InPar.2012.6339595>
 - [11] João Guerreiro, Aleksandar Ilic, Nuno Roma, and Pedro Tomás. 2019. GPU Static Modeling Using PTX and Deep Structured Learning. *IEEE Access* 7 (2019), 159150–159161. <https://doi.org/10.1109/ACCESS.2019.2951218>
 - [12] James Laros, Kevin Pedretti, Suzanne Kelly, Wei Shu, Kurt Ferreira, John Vandyke, and Courtenay Vaughan. 2013. *Energy Delay Product*. 51–55. https://doi.org/10.1007/978-1-4471-4492-2_8
 - [13] Etienne Le Sueur and Gernot Heiser. 2010. Dynamic Voltage and Frequency Scaling: The Laws of Diminishing Returns. In *2010 Workshop on Power Aware Computing and Systems (HotPower)*.
 - [14] Richard Schoonhoven, Bram Veenboer, Ben Van Werkhoven, and K. Joost Batenburg. 2022. Going green: optimizing GPUs for energy efficiency through model-steered auto-tuning. In *2022 IEEE/ACM International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*. 48–59. <https://doi.org/10.1109/PMBS56514.2022.00010>
 - [15] Ravid Shwartz-Ziv and Amitai Armon. 2022. Tabular data: Deep learning is not all you need. *Information Fusion* 81 (2022), 84–90. <https://doi.org/10.1016/j.inffus.2021.11.011>
 - [16] Jeffrey Spaan, Kuan-Hsun Chen, and Ana-Lucia Varbanescu. 2026. Reducing Compute Waste in LLMs through Kernel-Level DVFS. arXiv:2601.08539 [cs.PF] <https://arxiv.org/abs/2601.08539>
 - [17] Ben van Werkhoven. 2019. Kernel Tuner: A search-optimizing GPU code auto-tuner. *Future Generation Computer Systems* 90 (2019), 347–358. <https://doi.org/10.1016/j.future.2018.08.004>
 - [18] Daniel Velicka, Ondrej Vysocky, and Lubomir Riha. 2025. Methodology for GPU Frequency Switching Latency Measurement. In *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 830–839. <https://doi.org/10.1109/IPDPSW66978.2025.00133>
 - [19] Qiang Wang and Xiaowen Chu. 2020. GPGPU Performance Estimation With Core and Memory Frequency Scaling. *IEEE Transactions on Parallel and Distributed Systems* 31, 12 (2020), 2865–2881. <https://doi.org/10.1109/TPDS.2020.3004623>
 - [20] Qiang Wang, Laiyi Li, Weile Luo, Yijia Zhang, and Bingqiang Wang. 2024. DSO: A GPU Energy Efficiency Optimizer by Fusing Dynamic and Static Information. In *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*. 1–6. <https://doi.org/10.1109/IWQoS61813.2024.10682917>
 - [21] Zeyu Yang, Karel Adamek, and Wesley Armour. 2024. Accurate and Convenient Energy Measurements for GPUs: A Detailed Study of NVIDIA GPU's Built-In Power Sensor. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–17. <https://doi.org/10.1109/SC41406.2024.00028>
 - [22] Yijia Zhang, Qiang Wang, Zhe Lin, Pengxiang Xu, and Bingqiang Wang. 2024. Improving GPU Energy Efficiency through an Application-transparent Frequency Scaling Policy with Performance Assurance. In *Proceedings of the Nineteenth European Conference on Computer Systems (Athens, Greece) (EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 769–785. <https://doi.org/10.1145/3627703.3629584>